

Teknik Proje Raporu: İstanbul Neural-Guided A* Rota Planlayıcı

Hazırlayan: Enver Kutay Özbir, Gülbeyaz Vapur, Melike Eker

Tarih: 16 Haziran 2026

1. Projenin Amacı ve Kapsamı

Bu proje, İstanbul'un iki kıtalı ve asimetrik yol ağı topolojisinde yüksek performansla çalışan, çok duraklı (TSP) ve trafik farkındalıklı bir rota optimizasyon sistemi geliştirmeyi amaçlamaktadır. Klasik navigasyon sistemlerinin büyük veri setlerinde yaşadığı ölçeklenebilirlik sorununu çözmek amacıyla; hiyerarşik graf mimarisi, çift yönlü (bidirectional) arama ve yapay sinir ağı tabanlı (ONNX) sezgisel fonksiyonlar bir araya getirilmiştir. Sistem, uçtan uca çalışabilir bir REST API ve interaktif harita arayüzü ile tamamlanarak kullanıma hazır bir yazılım ürünü haline getirilmiştir.

2. Sistem Gereksinimleri ve Geliştirme Ortamı

Projenin bağımlılıkları ve çalışma ortamı modern makine öğrenmesi ve coğrafi bilgi sistemleri (CBS) standartlarına uygun olarak tasarlanmıştır.

Dil ve Çerçevesler: Python 3.10+, Flask (Web API)

Harita ve Graf İşleme: OSMnx ($\geq 1.9.0$), NetworkX, Shapely, GeoPandas

Makine Öğrenmesi: PyTorch (≥ 2.3), ONNX, ONNX Runtime

Görselleştirme: Folium, Leaflet.js

Donanım Uyumluluğu: Model eğitim süreçleri için Apple Silicon (M1) mimarisi üzerinde MPS (Metal Performance Shaders) donanım hızlandırması desteklenecek şekilde optimize edilmiştir.

3. Sistem Mimarisi ve Modüller

Proje, birbirine entegre çalışan modüler bir mimari üzerine inşa edilmiştir. Veri akışı; harita verisinin çekilmesinden, yapay zeka destekli hesaplamalara ve kullanıcı arayüzünde görselleştirmeye kadar doğrusal bir boru hattı (pipeline) şeklinde ilerler.

3.1. Veri Hazırlama ve Graf İşleme (`core/map_loader.py` & `graph_processor.py`)

Veri Çekimi: Overpass API kullanılarak sadece otoyol ve ana arterler çekilir. Mahalle içi düşük hiyerarşili yollar, arama uzayını küçültmek amacıyla bilinçli olarak hariç tutulmuştur.

SCC Temizliği: Ağ üzerindeki çıkmaz sokaklar ve tek yönlü ölü uçlar, Güçlü Bağlantılı Bileşen (Strongly Connected Component) analizi ile temizlenerek graf tutarlı hale getirilmiştir.

Hiyerarşik Katmanlama: Graf; 5 km altı şehir içi rotalar için "Arterial", 5 km üstü kıtalararası rotalar için "Express" katman olmak üzere ikiye ayrılmıştır.

3.2. Çekirdek Rota Motoru (**core/astar_engine.py**)

Sistemin ana yönlendirme motoru, Çift Yönlü Hiyerarşik A* algoritmasıdır.

Arama, başlangıçtan hedefe (ileri) ve hedeften başlangıca (geri) eşzamanlı yapılır.

Python'un standart **heapq** modülü kullanılarak *Lazy Deletion* stratejisi uygulanmış, bu sayede kuyruk güncellemelerinde $O(\log n)$ performansı garanti altına alınmıştır.

3.3. Sınır Ağı Entegrasyonu (**core/neural_heuristic.py & training/**)

Klasik mesafe ölçümü (Haversine) yerine, İstanbul coğrafyasını ve yol hiyerarşisini öğrenmiş bir Çok Katmanlı Algılayıcı (MLP - HeuristicNet) kullanılmıştır.

Özellik Vektörü (Feature Vector): Kavşak yoğunluğu (düğüm dereceleri), yol hiyerarşisi skoru ve coğrafi yönelim (bearing) gibi 8 boyutlu girdi kullanır.

ONNX Çıkarımı: Eğitilen model, PyTorch'un çalışma zamanı (runtime) hantallığından kurtulmak için INT8 kuantizasyonu ile ONNX formatına dönüştürülmüştür. Bu işlem, CPU üzerindeki model tahminlerini (inference) milisaniye altı seviyelere indirmiştir.

3.4. Çoklu Durak Planlayıcısı (**core/ring_planner.py**)

Kullanıcının seçtiği N adet durağı en uygun sıraya dizen TSP (Gezgin Satıcı Problemi) çözücü modüldür.

Asimetrik Maliyet Matrisi: NeuralHeuristic modülünün toplu tahmin (batch) özelliğiyle tüm noktalar arası maliyetler tek seferde hesaplanır.

Optimizasyon: Başlangıç rotası için *Nearest Neighbor* algoritması kullanılmış, ardından 2-*opt Local Search* ile rota iteratif olarak %5 hata payı içerisinde optimize edilmiştir.

3.5. Trafik Yönetim Sistemi (**core/traffic_manager.py**)

Gerçek zamanlı ve simüle edilmiş koşulları hesaba katan modüldür.

Zaman dilimlerine (sabah/akşam zirve saatleri) ve köprü geçişlerine özel temel çarpanlar uygulanır.

Rastgele olay simülasyonları (kaza, yol çalışması) üreterek ilgili yolların ağırlığını (edge cost) geçici olarak artırır. Opsiyonel olarak sistem TomTom Traffic API canlı veri entegrasyonuna da sahiptir.

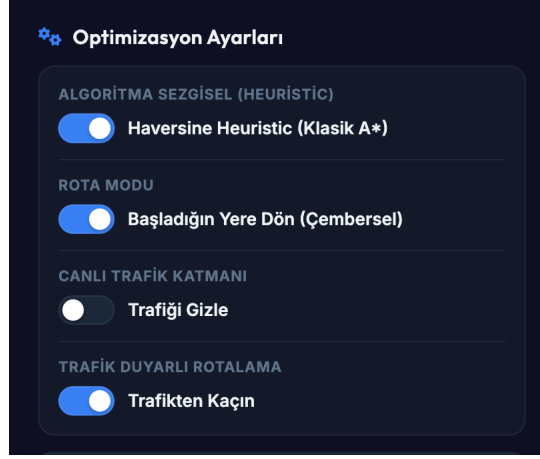
3.6. REST API ve Web Arayüzü (**app.py & templates/index.html**)

Sistem, Flask tabanlı bir web sunucusu üzerinden dış dünyaya açılır.

/api/route uç noktası (endpoint), kullanıcının girdiği JSON formatındaki durak listesini işler.

Ön yüzde Leaflet.js kullanılarak dönen poligon (polyline) koordinatları interaktif olarak çizilir.

4. Arayüz Kontrol Fonksiyonları ve Teknik İşlevleri



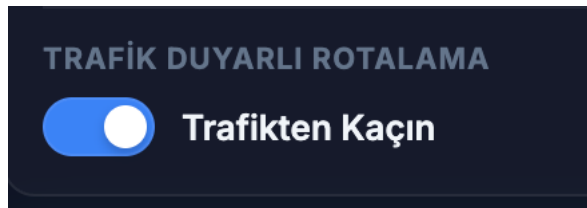
Kullanıcı arayüzünde (UI) yer alan dinamik kontrol anahtarları, arka plandaki yönlendirme algoritmasının çalışma parametrelerini gerçek zamanlı olarak manipüle eder:



4.1. Haversine Heuristic (Klasik A*) Anahtarı

İşlev: Algoritmanın "maliyet tahmin" (cost-to-go) yöntemini değiştirir.

Teknik Detay: Aktif edildiğinde sistem klasik Haversine formülü ile iki düğüm arasındaki kuş uçuşu mesafeyi hesaplar. Bu mod, sistemin temel performansını (baseline) test etmek için kullanılır. Kapatıldığında ise sistem, İstanbul coğrafyasını öğrenmiş olan **Neural Heuristic** (MLP) modelini devreye sokarak arama sırasında daha az düğüm genişletir ve hedefe daha akıllı yönelir.



4.2. Trafikten Kaçın (Traffic-Aware) Anahtarı

İşlev: Rota hesaplamasını "mesafe odaklı" olmaktan çıkarıp "zaman odaklı" hale getirir.

Teknik Detay: Aktif edildiğinde, graf üzerindeki kenar maliyetleri (edge weights) yalnızca fiziksel mesafe ile değil; anlık trafik yoğunluk gradyanı, zirve saat kısıtları ve simüle edilen kaza verileriyle çarpılarak güncellenir. Sistem, seyahat süresini minimize etmek için mesafece daha uzun fakat akıcı alternatif koridorları keşfeder.

5. Performans, Önbellekleme Stratejileri ve Rota Analizi

Sistem hızını artırmak ve tekrarlanan işlemleri önlemek için çok katmanlı bir önbellekleme mimarisi kurulmuştur:

Koordinat Önbelleği: Düğümün enlem/boylam bilgileri $O(1)$ erişim süresi için sözlük yapısında tutulur.

Mesafe Önbelleği: Hesaplanan düğümler arası Haversine mesafeleri `pickle` ile diske serialize edilerek sunucu başlangıç yükü azaltılır.

ONNX Tahmin Önbelleği: A^* 'ın aynı düğüm çiftlerini tekrar ziyaret etmesi durumunda modelin yeniden çalıştırılmasını engellemek için RAM üzerinde geçici bir sözlük kullanılır.



Sistem Performans Değerleri ve Rota Analiz Metrikleri Tablosu

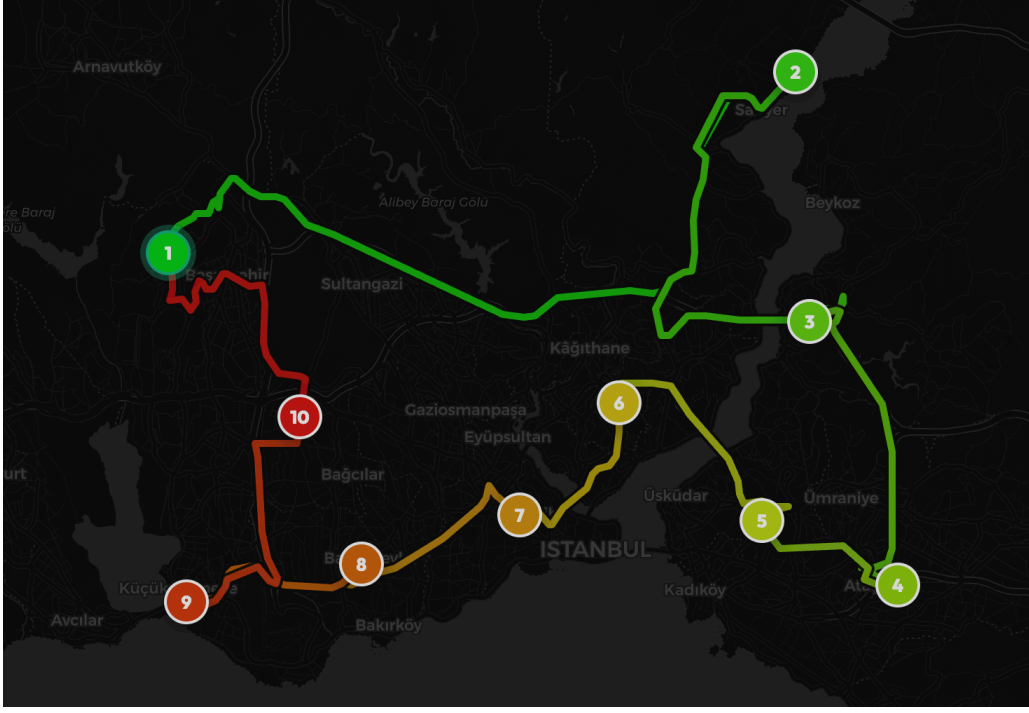
Örnek Metrikler:

Toplam Mesafe: 169.97 km

Tahmini Süre: 169 dk

İşlem Süresi (Inference): 501 ms

Segment Sayısı (Uğranılacak Konum): 10



Leaflet.js Arayüzü İstanbul Geneli Rota Haritası

Açıklama: 10 segmentlik optimizasyonu, asimetrik yol ayrımlarını ve trafik yoğunluk gradyanını (yeşilden kırmızıya geçiş) gösteren aktif harita ekran görüntüsü bu alana eklenecektir.

6. Sonuç

İstanbul Neural-Guided A* Rota Planlayıcı projesi, teorik yapay zeka algoritmalarının pratik bir yazılım mühendisliği ürününe nasıl entegre edilebileceğini başarıyla kanıtlamıştır. Model eğitiminden, algoritma optimizasyonuna ve arayüz entegrasyonuna kadar yazılım geliştirme döngüsünün tüm adımları tamamlanmıştır. Geliştirilen sistem, modüler altyapısı sayesinde gelecekte farklı coğrafyalara veya farklı sinir ağı mimarilerine kolayca adapte edilebilir yapıdadır.